Bahria University, Islamabad

Department of Software Engineering

Web Engineering Lab (Fall-2025)

Instructor: Engr. Subas Bilal

Student: Waleed Ahmed

Enrollment: 01-131232-093

Lab Journal: 10

Date: 4/22/2026

Lab # 12
Lab Task: Integrate Frontend and Backend for Complete Product CRUD Functionality

Full-Stack Integration Lab

Lab Objectives

By the end of this lab, students will be able to:

- Build a complete product management application using the MERN stack
- Connect a React frontend to a Node.js/Express backend via Axios
- Perform all CRUD operations: Create, Read, Update, and Delete
- Use React state and hooks to manage UI data dynamically
- Handle API responses and display real-time feedback to users
- Store and retrieve product data from MongoDB Atlas

Software / Tools Required

- Node.js (installed)
- Visual Studio Code
- MongoDB Atlas account
- Postman (for backend testing)
- A modern web browser (Chrome / Edge)

Lab Description

In this lab, you will integrate a React frontend with the Node.js/Express backend you built in a previous lab. The goal is to create a fully functional Product Management application that supports Create, Read, Update, and Delete (CRUD) operations. User interactions on the frontend will trigger HTTP requests to the backend API, which will read from and write to MongoDB Atlas.

References: https://www.youtube.com/watch?v=O3BUHwfHf84 | https://www.youtube.com/watch?v=98BzS5Oz5E4

Step 1: Project Structure Setup

Your MERN project folder should have the following structure:

```
NEW MERN
├── backend
│   ├── models
│   │   └── Product.js
│   ├── routes
│   │   └── products.js
│   ├── .env
│   └── server.js
└── frontend
    ├── src
    │   ├── components
    │   │   ├── ProductForm.jsx
    │   │   └── ProductList.jsx
    │   ├── App.jsx
    │   └── main.jsx
    └── package.json
```

If you do not already have a frontend folder, create it now:

```
cd NEW MERN
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios
```

Step 2: Backend – Product Model (models/Product.js)

Create a new file Product.js inside the backend/models folder. This defines the schema for a product stored in MongoDB.

```
import mongoose from 'mongoose';


const productSchema = new mongoose.Schema(
  {
    name:     { type: String, required: true },
    price:    { type: Number, required: true },
```

```
    category: { type: String, required: true },
    stock:    { type: Number, default: 0 },
  },
  { timestamps: true }
);


export default mongoose.model('Product', productSchema);
```

Ans. Done

Step 3: Backend – Product Routes (routes/products.js)

Create a new file products.js inside the backend/routes folder. This file implements all CRUD endpoints.

```
import express from 'express';
import Product from '../models/Product.js';


const router = express.Router();


// GET all products
router.get('/', async (req, res) => {
  try {
    const products = await Product.find();
    res.json(products);
  } catch (err) {
    res.status(500).json({ message: 'Server error' });
  }
});


// POST - create a new product
router.post('/', async (req, res) => {
  try {
    const product = new Product(req.body);
    await product.save();
```

```
    res.status(201).json(product);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// PUT - update a product by ID
router.put('/:id', async (req, res) => {
  try {
    const updated = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// DELETE - remove a product by ID
router.delete('/:id', async (req, res) => {
  try {
    await Product.findByIdAndDelete(req.params.id);
    res.json({ message: 'Product deleted' });
  } catch (err) {
    res.status(500).json({ message: 'Server error' });
  }
});

export default router;
```

Ans. Done

Step 4: Backend – Update server.js

Open your existing server.js and add the product routes and CORS middleware so the frontend can connect to it.

```
import express  from 'express';
import mongoose from 'mongoose';
import dotenv   from 'dotenv';
import cors     from 'cors';
import productRoutes from './routes/products.js';


dotenv.config();


const app = express();
app.use(cors());           // allow requests from React frontend
app.use(express.json());


app.use('/api/products', productRoutes);


mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(console.error);


app.listen(process.env.PORT || 5000, () =>
  console.log('Server running on port 5000'));
```

Install the cors package in your backend folder:

```
npm install cors
```

Ans. Done

Step 5: Test Backend Endpoints with Postman

Before building the frontend, verify all four endpoints work correctly in Postman.

Create a Product (POST)

- Method: POST
- URL: http://localhost:5000/api/products
- Body (JSON):

```
{
  "name": "Laptop",
  "price": 75000,
  "category": "Electronics",
  "stock": 10
}
```

Get All Products (GET)

- Method: GET
- URL: http://localhost:5000/api/products

Update a Product (PUT)

- Method: PUT
- URL: http://localhost:5000/api/products/<product_id>
- Body (JSON): any fields to update, e.g. { "price": 80000 }

Delete a Product (DELETE)

- Method: DELETE
- URL: http://localhost:5000/api/products/<product_id>

Ans. Done

Step 6: Frontend – ProductForm Component

Create the file frontend/src/components/ProductForm.jsx. This form allows users to add new products or update existing ones.

```
import { useState, useEffect } from 'react';
import axios from 'axios';


const API = 'http://localhost:5000/api/products';
```

```
export default function ProductForm({ onSave, editProduct, setEditProduct }) {
  const [form, setForm] = useState(
    { name: '', price: '', category: '', stock: '' }
  );

  useEffect(() => {
    if (editProduct) setForm(editProduct);
  }, [editProduct]);

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (editProduct) {
      await axios.put(`${API}/${editProduct._id}`, form);
      setEditProduct(null);
    } else {
      await axios.post(API, form);
    }
    setForm({ name: '', price: '', category: '', stock: '' });
    onSave();
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>{editProduct ? 'Edit Product' : 'Add Product'}</h2>
      <input name='name'     placeholder='Name'     value={form.name}
onChange={handleChange} required />
      <input name='price'    placeholder='Price'    value={form.price}
onChange={handleChange} required />
      <input name='category' placeholder='Category' value={form.category}
onChange={handleChange} required />
```

```
      <input name='stock'    placeholder='Stock'    value={form.stock}
onChange={handleChange} />

      <button type='submit'>{editProduct ? 'Update' : 'Add'}</button>

      {editProduct && (

        <button type='button' onClick={() => setEditProduct(null)}>Cancel</button>

      )}

    </form>

  );

}
```

Ans. Done

Step 7: Frontend – ProductList Component

Create the file frontend/src/components/ProductList.jsx. This component fetches and displays all products, and provides Edit and Delete buttons for each.

```
import axios from 'axios';



const API = 'http://localhost:5000/api/products';



export default function ProductList({ products, onDelete, onEdit }) {

  const handleDelete = async (id) => {

    await axios.delete(`${API}/${id}`);

    onDelete();

  };



  return (

    <div>

      <h2>Product List</h2>

      {products.length === 0 && <p>No products found.</p>}

      {products.map((p) => (

        <div key={p._id} style={{ border: '1px solid #ccc', margin: '8px', padding:
'8px' }}>

          <strong>{p.name}</strong> - Rs. {p.price}
```

```
          <br />
          Category: {p.category} | Stock: {p.stock}
          <br />
          <button onClick={() => onEdit(p)}>Edit</button>
          <button onClick={() => handleDelete(p._id)}>Delete</button>
        </div>
      ))}
    </div>
  );
}
```

Ans. Done

Step 8: Frontend – App.jsx (Main Integration)

Replace the contents of frontend/src/App.jsx with the following. This is the root component that ties everything together.

```
import { useState, useEffect } from 'react';
import axios from 'axios';
import ProductForm from './components/ProductForm';
import ProductList from './components/ProductList';

const API = 'http://localhost:5000/api/products';

export default function App() {
  const [products,    setProducts]    = useState([]);
  const [editProduct, setEditProduct] = useState(null);

  const fetchProducts = async () => {
    const res = await axios.get(API);
    setProducts(res.data);
  };
```

```
  useEffect(() => { fetchProducts(); }, []);


  return (
    <div style={{ maxWidth: '700px', margin: '0 auto', padding: '20px' }}>
      <h1>Product Manager (MERN)</h1>
      <ProductForm
        onSave={fetchProducts}
        editProduct={editProduct}
        setEditProduct={setEditProduct}
      />
      <ProductList
        products={products}
        onDelete={fetchProducts}
        onEdit={setEditProduct}
      />
    </div>
  );
}
```

Ans. Done

Step 9: Running the Full Application

You need two terminals running simultaneously – one for the backend and one for the frontend.

Terminal 1 – Start Backend

```
cd NEW MERN/backend

npm run dev
```

Expected output:

```
MongoDB connected

Server running on port 5000
```

Terminal 2 – Start Frontend

```
cd NEW MERN/frontend

npm run dev
```

Expected output:

```
  VITE v5.x.x  ready in xxx ms

  ➜  Local:   http://localhost:5173/
```

Open http://localhost:5173 in your browser. You should see the Product Manager interface.

Ans. Done

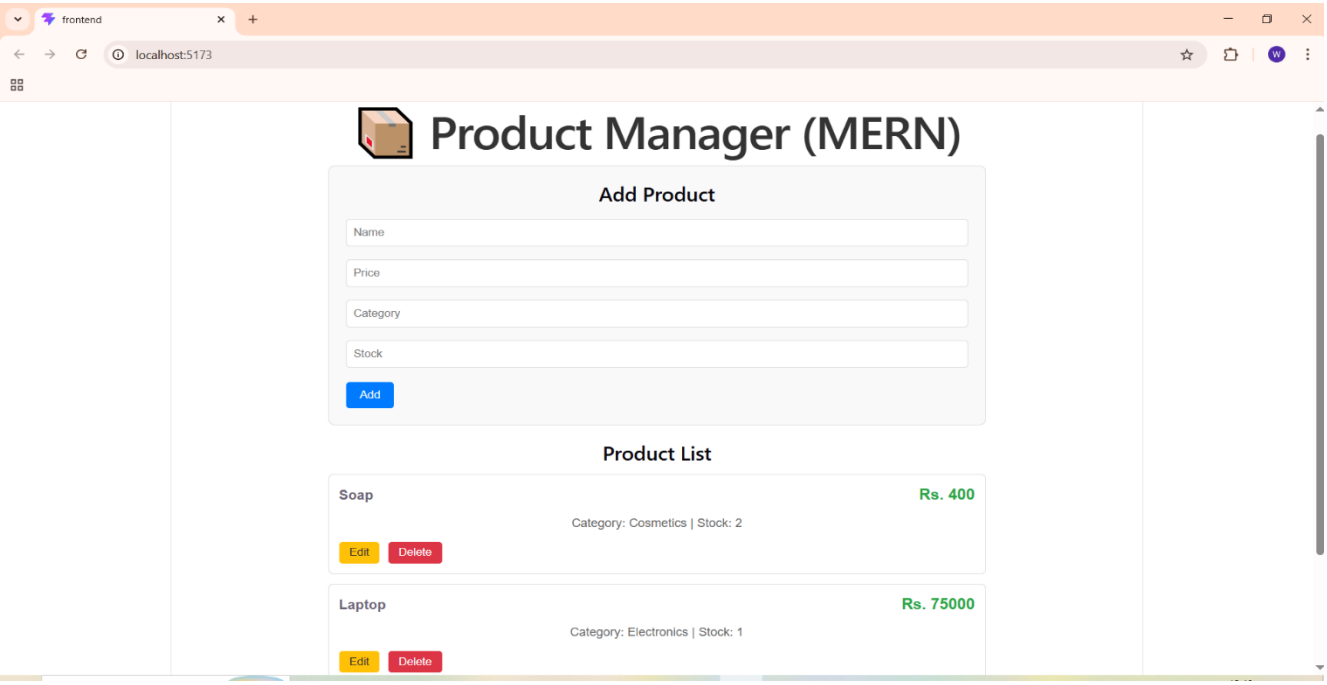
Step 10: Testing CRUD in the Browser

Perform all four CRUD operations through the browser UI and verify they work end-to-end.

Solutions:

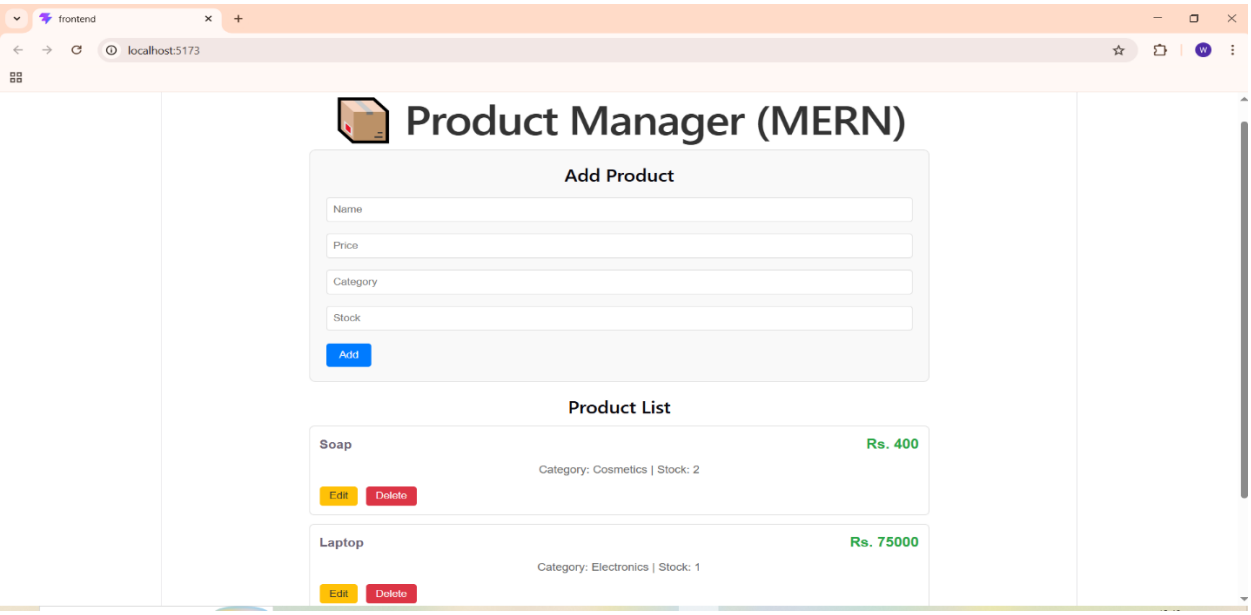
Create

Fill in the Add Product form with a name, price, category, and stock value. Click Add. The product should appear in the list below without refreshing the page.



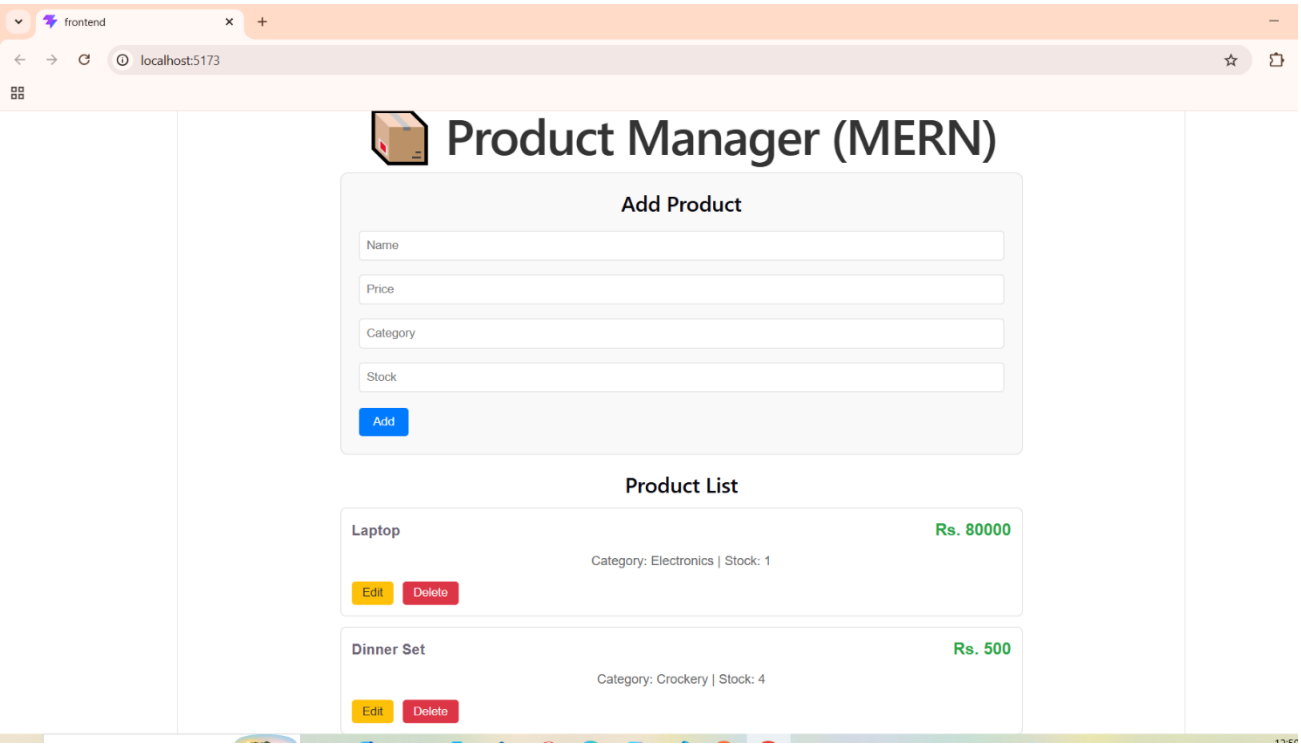
Read

Reload the page. All previously added products should still be displayed (data persisted in MongoDB).



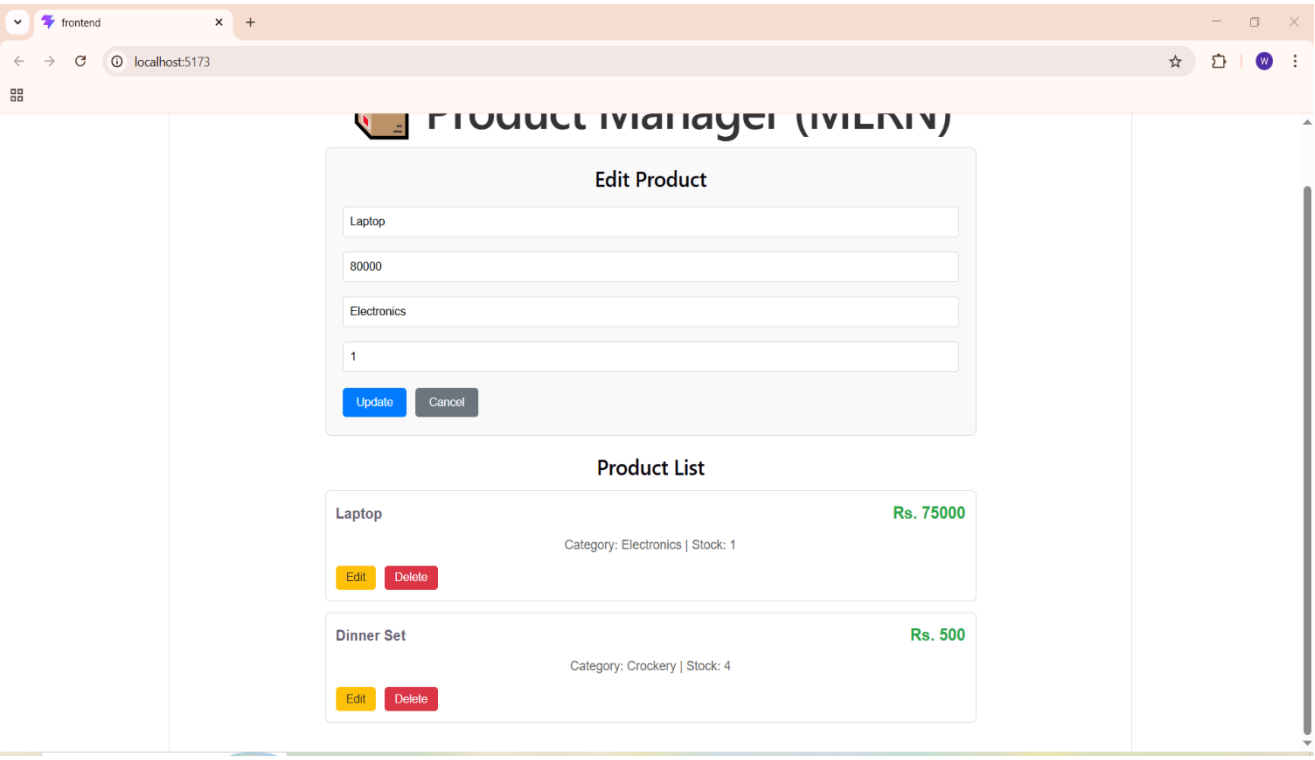
Delete

Click the Delete button on any product. It should be removed from the list immediately.



Update

Click the Edit button on any product. The form will populate with the product's current data. Change a value and click Update. The list should refresh with the new values.



Step 11: MongoDB Atlas Verification

1. Open your MongoDB Atlas account.
2. Navigate to your cluster and click Browse Collections.
3. Select your database and find the products collection.
4. Confirm that the documents match the products shown in the browser.

Solutions:

```
_id: ObjectId('69e7d4582fca007a57876d00')
name : "Laptop"
price : 80000
category : "Electronics"
stock : 1
createdAt : 2026-04-21T19:47:36.354+00:00
updatedAt : 2026-04-21T19:50:16.640+00:00
__v : 0
```

```
_id: ObjectId('69e7d47c2fca007a57876d03')
name : "Dinner Set"
price : 500
category : "Crockery"
stock : 4
createdAt : 2026-04-21T19:48:12.408+00:00
updatedAt : 2026-04-21T19:48:12.408+00:00
__v : 0
```

```
_id: ObjectId('69e7d65f2fca007a57876d0c')
name : "Soap"
price : 400
category : "Cosmetics"
stock : 2
createdAt : 2026-04-21T19:56:15.649+00:00
```

Expected Learning Outcomes

Students will be able to:

- Integrate a React frontend with a Node.js/Express backend
- Use Axios to send HTTP requests (GET, POST, PUT, DELETE) from React
- Manage frontend state using useState and useEffect hooks
- Implement full CRUD for a real-world data model in a MERN application
- Verify data persistence using MongoDB Atlas

Lab Submission

- Screenshot of the browser showing the product list (at least 3 products)
- Screenshot of each CRUD operation (create, update, delete) working in the browser
- Screenshot of the MongoDB Atlas products collection
- Zip file of the complete project (both backend and frontend folders)
- All screenshots compiled into a single PDF